# Práctica 2

**Metaheurísticas**

**Juan Antonio Mellado Arenas**
**Nicolás Sánchez Álvarez**
**Antonio Jesús Merlo Morales**
**Sergio Palacios López**
**Sergio García Atanasio**

**Universidad de Córdoba**

# Índice


**1 Introducción** **2**

**2 Enunciado de la práctica** **2**

**3 Representación** **2**

**4 Población inicial** **3**

**5 Selección padres** **4**

**6 Cruces entre individuos** **4**

**7 Mutación de individuos** **6**

**8 Reemplazo** **6**

**9 Probabilidades de mutación y cruce** **6**

**10 Criterios de parada** **8**

**11 Consideraciones extra** **8**
11.1 Control de dominios . . . . . 8
11.2 Normalización y cálculo de distancias . . . . . 9

**12 Random Search, Grid Search y Algoritmo genético** **10**
12.1 Random Search . . . . . 10
12.2 Grid Search . . . . . 10

**13 Análisis resultados** **11**

**14 Conclusión** **11**

# 1 Introducción

En este documento, se muestran sendos proceso y resultado del desarrollo de la segunda práctica de la asignatura de Metaheurísticas. El objetivo de esta práctica consiste en la implementación de un algoritmo genético para la optimización de hiperparámetros de un modelo de aprendizaje atomático, así como su comparación entre los algoritmos Random Search y Grid Search. Cualquier aspecto relativo al código fuente, o información adicional queda en el repositorio de la práctica.

# 2 Enunciado de la práctica

La práctica plantea que, dado un conjunto real de datos (será el dataset Wine Quality), se ha de maximizar la precisión de clasificación del modelo de aprendizaje automático dado. Wine Quality considera datos como pH del vino, su acidez, densidad, entre otros. Sobre esto, se transforma la clasificación del problema tal que sea de carácter binario, resultando en que un vino puede ser bueno o malo según si su calidad supera o no supera 6.

Sobre el modelo Random Forest no se debe tomar consideraciones en cuanto a su funcionamiento por dentro, ya que, el objeto de la práctica es optimizar los hiperparámetros, y eso incluye una abstracción del modelo tal que se perciba como una herramienta capaz de evaluar esos parámetros y devolviendo una calidad.

Así pues, para lo que será el algoritmo genético, tenemos una codificación tal que un individuo es un vector con sus genes (parámetros) en orden. Es mixto ya que algunos genes tienen un dominio binario, mientras que otros operan en un rango definido para los números reales.

Utilizando un paradigma orientado a objetos, se ha implementado la clase Individual, que tiene los genes como atributos, además de algunos campos ajenos a los genes que serán útiles a la hora de implementar el algoritmo, como la posición que ocupa cada individuo en la población. Haciendo uso del modelo, se obtiene un atributo score o puntuación, que es la media de los 5 folds como se indica en la práctica (de nuevo, esto no es crucial en el desarrollo del algoritmo). Posteriormente se compara con 6 para decidir si la calidad es apta o no. Una población es un conjunto de objetos de la clase individuo. Con la representación definida ya se pueden empezar a plantear las diferentes etapas del algoritmo genético.

# 3 Representación

Lo primero que haremos será definir la representación que utilizaremos para aplicar nuestro algoritmo genético y abordar nuestro problema principal. La representación más sencilla que podemos utilizar es una de tipo lineal, en este caso, combinada con valores enteros y binarios. Para aplicarlo en Python, crearemos una clase con los siguientes campos.

Estos son los atributos que aparecen en nuestra clase individuo, a efectos prácticos trabajaremos con ella como si fuese un vector de parámetros. Los dos últimos parámetros, position y score, no se utilizarán a la hora de realizar operaciones de selección, cruce o mutación.

El parámetro position indica la posición que tiene cada individuo en la población, cosa que será bastante útil cuando necesitemos hacer reemplazamiento. El parámetro score indica el valor que devuelve la función de fitness, y se calcula únicamente cuando se crea el individuo para ahorrar cálculos innecesarios (también se recalcula cuando se hace una operación de mutación, por ejemplo).

```
class Individual:
  def __init__(
      self,n_estimators,max_depth,min_samples_split,min_samples_leaf,max_features,
  bootstrap,
      criterion,class_weight,max_leaf_nodes,min_impurity_decrease,random_state,
  position,
  ):
      self.n_estimators = n_estimators
      self.max_depth = max_depth
      self.min_samples_split = min_samples_split
      self.min_samples_leaf = min_samples_leaf
      self.max_features = max_features
      self.bootstrap = bootstrap
      self.criterion = criterion
      self.class_weight = class_weight
      self.max_leaf_nodes = max_leaf_nodes
      self.min_impurity_decrease = min_impurity_decrease
      self.random_state = random_state
      self.position = position
      self.score = model.evaluate_solution(self)
```

Listing 1: Constructor de la clase Individuo

# 4 Población inicial

El comienzo del algoritmo genético es, la creación de una población inicial sobre la que iterar el resto de las etapas de este. Para el algoritmo se ha implementado una generación de población inicial secuencial. Esta consiste en crear individuos aleatorios, con el matiz de que los individuos distan entre sí una cantidad considerable de diferencias en sus genes. Al final resultará una población de la que partir con individuos que tienen genes aleatorios, pero que, de manera general son todos diferentes entre sí como mínimo una distancia definida.

Para implementar este método, se han normalizado los genes entre 0 y 1. Posteriormente se hace uso de la distancia euclidiana, lo que permite definir como de alejados se encuentrán dos individuos según sus genes. Para más información, mirar la referencia a

Se ha elegido esta técnica ya que si se tiene en cuenta cualidades como la diversidad de la población inicial, la generación secuencial permite partir de un conjunto extenso y que cubre un espacio amplio de individuos. El costo de implementarlo y ejecutarlo a nivel computacional es medio, y su punto más débil es que las soluciones iniciales no serán de gran calidad (al estar creados los individuos de manera aleatoria). Con todo esto sigue siendo un método idóneo por su balance y, teniendo en cuenta que la mejora de soluciones es algo que se obtendrá posteriormente en el algoritmo, y que, en este punto interesa diversificar lo máximo posible.

# 5 Selección padres

Una vez que se tiene la población inicial, el próximo paso es escoger los padres de la población. Para esto el grupo ha decidido hacer uso de la selección por torneo. Esta técnica, es una de las más costosas y complejas, no obstante, resulta en mejores soluciones de forma general. Además, es un balance entre aleatoriedad y una selección de los mejores individuos.

La cuestión principal en selección por torneo es, el tamaño del torneo. Si se toma un tamaño muy grande surge alta presión selectiva, y al final únicamente pasan los individuos mejores, de forma repetida. Esto puede terminar en muy poca diversificación y en incesto.

Por eso nuestro tamaño de torneo es 3. Se seleccionan tres individuos aleatorios de la población, y se evalúan según su score, el individuo que más score tenga será el seleccionado para ser padre. La selección de individuos para enfrentarlos en el torneo es con reposición (pueden aparecer individuos repetidos)

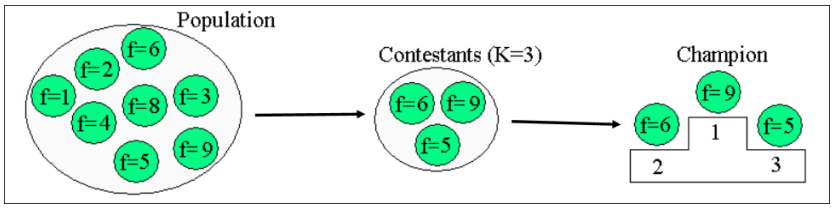


Figure 1: Selección por torneo, con K=3

En adición, se introduce un concepto nuevo que ayudará a mejorar la media de la puntuación de una población. El elitismo, consiste en seleccionar como padres a un conjunto de individuos (la élite) porque son aquellos con mayor puntuación, saltándose todo el proceso de selección.

Se ha implementado que además de los padres elegidos por torneo, los dos mejores individuos de cada generación sobrevivan y pasen a la próxima generación. Este fenómeno contribuye a la mejora de la media de la puntuación del algoritmo.

# 6 Cruces entre individuos

Una vez escogidos los padres de la población, la etapa que sigue consiste en cruzar dichos individuos para obtener nuevos individuos hijos. Para efectuar este proceso, se ha desarrollado un operador de cruce híbrido, que combina dos estrategias distintas: cruce uniforme y cruce en n puntos.
El cruce uniforme consiste en generar un nuevo individuo seleccionando, para cada gen, el valor de uno de los dos progenitores de forma independiente y aleatoria. De este modo, cada parámetro del hijo proviene de uno de los padres con una cierta probabilidad, favoreciendo la exploración del espacio de soluciones.

Por otro lado, el cruce en n puntos es una técnica en la que se seleccionan varios puntos de corte a lo largo del cromosoma. A partir de estos puntos, se alternan bloques de genes entre ambos progenitores para construir el descendiente. En este caso se han utilizado 4 puntos de corte. Este método permite preservar combinaciones parciales de parámetros que pueden ser beneficiosas, favoreciendo la explotación.

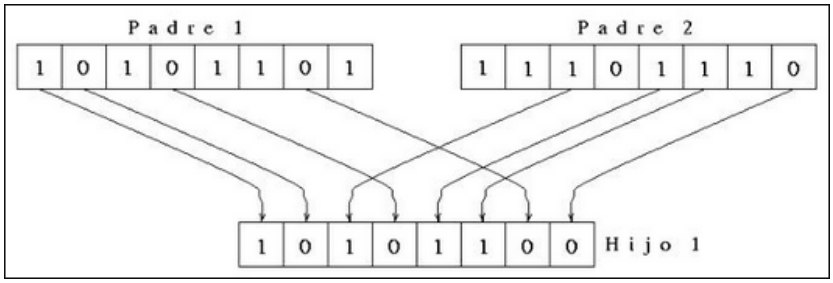


Figure 2: Cruce uniforme

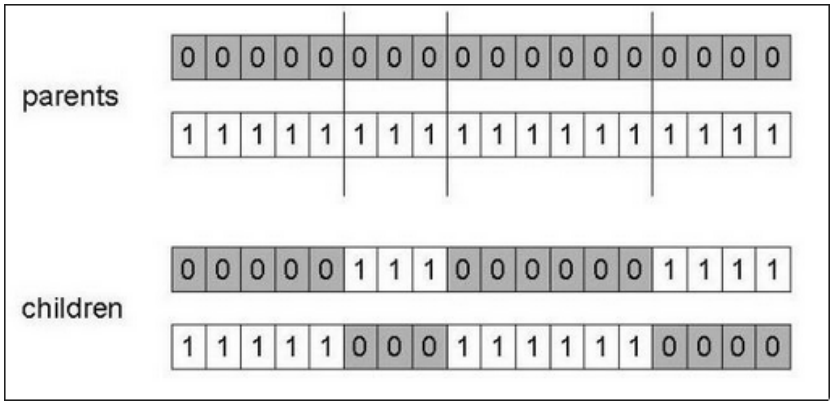


Figure 3: Cruce de n-puntos

Ambos operadores generan dos descendientes a partir de cada pareja de padres.
El algoritmo activa el cruce si la probabilidad de cruce lo permite (este mecanismo se detalla más adelante en el documento), y en caso afirmativo, se selecciona de manera aleatoria cuál de los dos operadores aplicar, asignando aproximadamente la misma probabilidad a cada uno. De este modo, la mitad de los cruces se realizan mediante cruce uniforme y la otra mitad mediante cruce en n puntos.

Las fortalezas del cruce híbrido frente a únicamente utilizar uno de los dos mecanismos explicados, son especialmente respecto al equilibrio de exploración y explotación, además, el algoritmo ahora tiene más variabilidad, lo que reduce la convergencia prematura. Sus debilidades son principalmente el costo ya que hay más evaluaciones, y al hacer dos hijos por cruce se deben entrenar todos estos, causando que el algoritmo tarde más. Además, la implementación ha resultado más compleja que utilizar un cruce úni

Para implementar este método, se han normalizado los genes entre 0 y 1. Posteriormente se hace uso de la distancia euclidiana, lo que permite definir como de alejados se encuentrán dos individuos según sus genes. Para más información, revisar el punto de "Consideraciones extra"9.

Se ha elegido esta técnica ya que si se tiene en cuenta cualidades como la diversidad de la población inicial, la generación secuencial permite partir de un conjunto extenso y que cubre un espacio amplio de individuos. El costo de implementarlo y ejecutarlo a nivel computacional es medio, y su punto más débil es que las soluciones iniciales no serán de gran calidad (al estar creados los individuos de manera aleatoria). Con todo esto sigue siendo un método idóneo por su balance y, teniendo en cuenta que

la mejora de soluciones es algo que se obtendrá posteriormente en el algoritmo, y que, en este punto interesa diversificar lo máximo posible. generación. Esteco.

# 7 Mutación de individuos

La mutación consiste en modificar ligeramente ciertos individuos, en este caso se aplica una mutación uniforme, aunque, se va aplicando a todos los genes del individuo. El operador recorre cada gen con una probabilidad de modificarlo, y si esta acierta, se cambia el gen de manera aleatoria (dentro del dominio permitido de cada parámetro o gen). Para garantizar que si se activa la mutación el individuo va a cambiar de manera segura, (podría recorrer todos los genes sin varíar ninguno), el operador asegura que al menos un gen cambia aleatoriamente. Tras la mutación, se actualiza el score del individuo.

Esta estrategia resulta poderosa para introducir nuevos valores que antes no estaban en la población, reduciendo convergencia prematura y explorando nuevas zonas. De hecho, si la población sufre estancamiento, puede saltar a soluciones mejores que no existirían solo con el cruce. A diferencia del cruce, se ha optado por un operador único y más simple. La desventaja principal es que podría romper alguna solución con puntuación muy elevada.

# 8 Reemplazo

Tras la mutación, el algoritmo genético realiza la técnica de reemplazo del peor individuo (o individuos). Este ordena primero la población según el valor obtenido al evaluar la solución, y dependiendo del número de individuos que vayan a ser reemplazados en la población, se seleccionan aquellos individuos para su reemplazo.

En este caso, se ha tenido en cuenta si la población necesita diversificar más o menos. Para cuando es necesario intensificar, se reemplazarán a los n peores de la población con los mejores hijos obtenidos, pero en el caso de que sea necesario diversificar más, se reemplazará a los n peores de la población con un individuo aleatorio, para que existan individuos más diferentes.

Se pensó en un primer momento en utilizar crowding determinístico ya que es muy útil a la hora de mantener la diversidad en la población, pero no nos logró convencer, ya que en nuestro caso no llegaba a converger lo rápido que queríamos, y la varianza no variaba mucho, por lo siempre aumentaba o disminuía una de las probabilidades de cruce o de mutación (siguiente apartado).

# 9 Probabilidades de mutación y cruce

Se ha tenido en cuenta la variabilidad de la probabilidad de mutación y la probabilidad de cruce. Para ello, el algoritmo calcula por cada generación la media de score de toda la población, y la varianza de los parámetros de los individuos que se encuentran en la población, normalizando los individuos como se ha hecho anteriormente para la creación de la población.

No hemos utilizado la varianza del score de la población, ya que realmente un individuo con unos parámetros diferentes al de otro individuo puede tener el mismo score. De esta forma, comprobamos realmente la variabilidad que existe en la población.

Primero hemos comenzado inicializando las probabilidades de mutación y de cruce a 0.5. Después, en cada generación dependiendo de un valor de varianza predefinido (probando, hemos determinado que en torno a 0.12 es un valor idóneo).

- Si la varianza es menor que este valor predefinido, significa que los individuos de la población son muy parecidos entre ellos, por lo que necesitamos diversificar, y con ello aumentar la probabilidad de mutación y disminuir la de cruce.
- Si la varianza es mayor que el valor de threshold, significa que los individuos son lo suficientemente diferentes como para poder intensificar, así que aumentaremos el valor de la probabilidad de cruce y disminuiremos la de mutación

Dicho esto, lo usual es que al principio vaya aumentando mucho la probabilidad de cruce, hasta que llegue un punto que la varianza es tan baja, que debería empezar a aumentar la probabilidad de mutación y disminuir la de cruce, hasta llegar a un tope. Un ejemplo de una ejecución se puede observar en la siguiente gráfica:

| iteraciones | media˙score | varianza˙params | prob˙mut | prob˙cross |
|---|---|---|---|---|
| 1 | 0.7028140510710553 | 0.13105116796307203 | 0.4 | 0.6 |
| 2 | 0.7289481289184954 | 0.13884470396281984 | 0.30000000000000004 | 0.7 |
| 3 | 0.7371556622257053 | 0.128308289906175 | 0.20000000000000004 | 0.7999999999999999 |
| 4 | 0.7372606452455589 | 0.1160213569315124 | 0.30000000000000004 | 0.7 |
| 5 | 0.7292331178160921 | 0.12611999786122535 | 0.20000000000000004 | 0.7999999999999999 |
| 6 | 0.73361391392372 | 0.10913515483665139 | 0.30000000000000004 | 0.7 |
| 7 | 0.7295509241118078 | 0.14706005928755064 | 0.20000000000000004 | 0.7999999999999999 |
| 8 | 0.7328609913793103 | 0.10205941479743767 | 0.30000000000000004 | 0.7 |
| 9 | 0.73048891392372 | 0.1020139208578844 | 0.4 | 0.6 |
| 10 | 0.7328066222570532 | 0.08846796140221659 | 0.5 | 0.5 |
| 11 | 0.7249912650208987 | 0.11329043457262836 | 0.6 | 0.4 |
| 12 | 0.7241814426593521 | 0.12084454595059614 | 0.5 | 0.5 |
| 13 | 0.7235560344827586 | 0.12232828585044904 | 0.4 | 0.6 |
| 14 | 0.7264472309299895 | 0.12106574331028173 | 0.30000000000000004 | 0.7 |
| 15 | 0.7350957582288403 | 0.09981025852975751 | 0.4 | 0.6 |
| 16 | 0.7284006008359457 | 0.10386981148872482 | 0.5 | 0.5 |
| 17 | 0.7285570957419018 | 0.10892883376974954 | 0.6 | 0.4 |
| 18 | 0.7223534646029258 | 0.12170766111266987 | 0.5 | 0.5 |
| 19 | 0.727879032784744 | 0.10586170175849068 | 0.6 | 0.4 |
| 20 | 0.7236584051724139 | 0.11310044956276966 | 0.7 | 0.30000000000000004 |

Podemos observar, como en las últimas iteraciones va aumentando la probabilidad de mutación y disminuyendo la de cruce, como habíamos predicho.

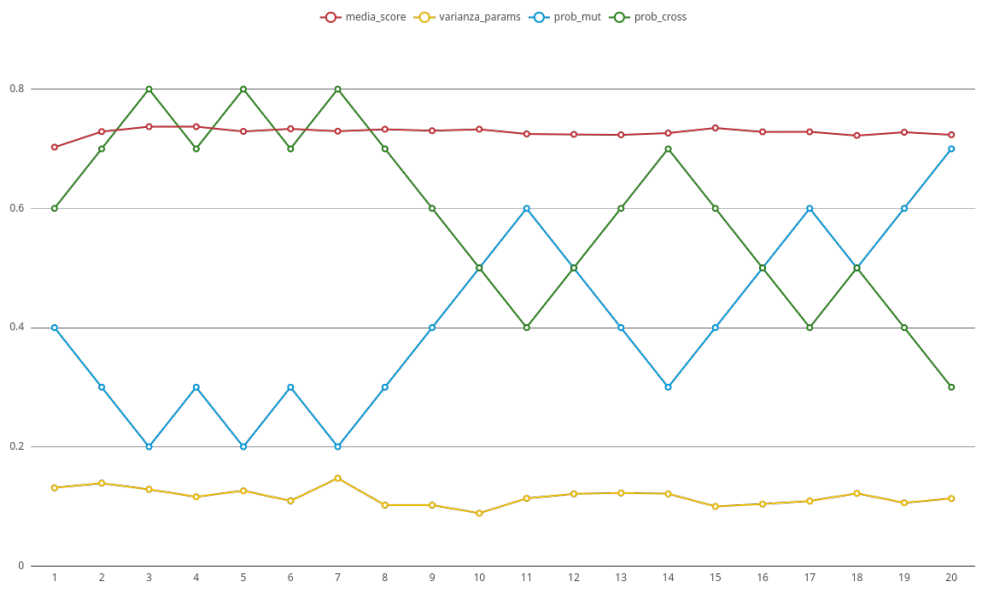


Figure 4: Probabilidades de mutación y cruce tras 20 iteraciones

# 10 Criterios de parada

Se han considerado los siguientes criterios para detener la evolución de la población:

- **Número máximo de generaciones**: Para evitar que se ejecute en exceso, en el algoritmo genético en cuestión se considera un máximo de 24 generaciones. Se ha observado que este número de iteraciones es suficiente como para inferir conclusiones sin suponer demasiado tiempo de ejecución.
- **Inversión de probabilidades**: Además del límite físico de 24 iteraciones, se ha implementado un criterio de parada basado en la convergencia evolutiva que detiene el algoritmo cuando se da la inversión de probabilidades de operadores $P_{mut} > P_{cross}$.

# 11 Consideraciones extra

## 11.1 Control de dominios

Se han de tener en cuenta los valores de los parámetros de cada individuo y que se mantengan dentro de su dominio correspondiente en los cruces y las mutaciones.

Debido a que los cruces implementados no modifican los valores de los parámetros, no hay que tomar ninguna consideración de seguridad en este aspecto.

La única mutación implementada es la uniforme, se garantiza que los valores se mantengan dentro de los dominios definiendo el rango de la función de generación aleatoria.

## 11.2 Normalización y cálculo de distancias

Para que el algoritmo gestione la diversidad de manera cuantitativa, se ha implementado un sistema de normalización en el rango $[0, 1]$ para cada gen del individuo.

Debido a que los hiperparámetros del modelo operan en escalas distintas, una distancia calculada sobre los valores reales daría un peso excesivo a las variables con más rango. La función `NormaliseIndividual` transforma cada gen para normalizarlo:

- **Variables numéricas de rango definido**: Se escalan dividiendo el valor por el límite máximo permitido.
- **Variables categóricas y booleanas**: Se mapean directamente a 0.0 o a 1.0.
- **Manejo de valores especiales**: Se mapean a valores intermedios (`"sqrt" -> 0.5, "log2" -> 0.3`) para permitir su comparación matemática.

```python
def NormaliseIndividual(ind):
  # Convertir max_features
  mf = ind.max_features
  if mf is None:
      mf_norm = 0.0
  elif isinstance(mf, str):
      if mf == "sqrt":
          mf_norm = 0.5
      elif mf == "log2":
          mf_norm = 0.3
      else:
          mf_norm = 0.0
  else:
      # ya es float (debe estar en [0,1])
      mf_norm = float(mf)

  # bootstrap ya es booleano 0/1
  bootstrap_norm = 1.0 if ind.bootstrap else 0.0

  # criterion y class_weight ya son 0/1, pero aseguramos float
  criterion_norm = float(ind.criterion)
  class_weight_norm = float(ind.class_weight)

  return np.array(
      [
          ind.n_estimators / 300.0,
          ind.max_depth / 30.0,
          ind.min_samples_split / 20.0,
          ind.min_samples_leaf / 20.0,
          mf_norm,
          bootstrap_norm,
          criterion_norm,
          class_weight_norm,
          ind.max_leaf_nodes / 200.0,
          ind.min_impurity_decrease / 0.1,
          ind.random_state / 1000.0,
```

```
        ]
    )
```

Listing 2: Función de normalización de individuo

# 12 Random Search, Grid Search y Algoritmo genético

En adición al algoritmo genético, se han implementado una búsqueda aleatoria y grid search para comparar ambos métodos. De esta forma, podremos observar cómo se comporta nuestro algoritmo frente a opciones más clásicas, y si de alguna manera es mejor o peor.

## 12.1 Random Search

La búsqueda aleatoria consiste en generar individuos (soluciones) de manera estocástica, tal que por cada gen (parámetro) se inicializa en un valor aleatorio. Este proceso se itera repetidas veces.

Este método es de los más rápidos que hay, pero como observaremos a continuación, la gran desventaja que tiene es la inconsistencia de los datos obtenidos por cada ejecución del algoritmo. Como resultado, la varianza debería ser bastante más elevada.

## 12.2 Grid Search

Grid search es una técnica de búsqueda exhaustiva de los parámetros objeto del problema. Se definen unas divisiones (grid) con los posibles valores de los parámetros, y sobre estos valores se generan combinaciones.

En este caso, se ha definido una lista de la división de esos parámetros con los posibles valores:

**n_estimators** {50, 100, 200}

**max_depth** {5, 10, 20, None}

**min_samples_split** {2, 5, 10}

**min_samples_leaf** {1, 2, 4}

**max_features** {*sqrt*, *log2*, None}

**bootstrap** {True, False}

**criterion** {*gini*, *entropy*}

Sobre estas opciones se crean los individuos, para finalmente ordenarlos y obtener las mejor o mejores configuraciones posibles. La principal bondad es que dentro del espacio (grid) que hemos definido, se afinan soluciones muy buenas (se encuentra la mejor combinación), aunque, la decisión surge al elegir qué se considera como división ya que grid search es computacionalmente muy costoso, y podría llegar a ser inviable.

Esto es fundamental para comprender los resultados así como la comparativa con los otros dos métodos relativos a esta práctica.

## 13 Análisis resultados

## 14 Conclusión